

Pandas Cheat Sheet

Personal notes — core concepts & interesting points

1. SERIES

Creating a Series

```
import pandas as pd
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
```

Arithmetic — Mismatched Indexes

When two Series have different indexes, pandas aligns on common keys and fills NaN for unmatched ones — no crash.

```
s1 = pd.Series([20, 30, 40, 50], ['a', 'b', 'c', 'd'])
s2 = pd.Series([10, 5, 3, 11], ['a', 'c', 'f', 'g'])
result = s1 + s2 # NaN for keys not shared by both
```

2. DATAFRAME BASICS

Creating a DataFrame

```
data = {'Name': ['Ali', 'Atlas', 'Lena'], 'Age': [25, 30, 22 ], 'Salary': [3000, 4500, 2800 ]}
df = pd.DataFrame(data)
```

Selecting Columns

Use double brackets `[[ ]]` to get a DataFrame back — single bracket returns a Series.

```
df['Age'] # Series
df[['Age', 'Salary']] # DataFrame
```

Adding / Modifying Columns & Rows

```
df['Bonus'] = 500
df['Extra'] = df['Salary'] * 0.1
df.loc['new_row'] = [35, 'Bob', 5000, 600]
```

Selecting Rows — .iloc vs .loc

```
df.iloc[0] # first row by position
df.iloc[0:3] # rows 0-2
df.iloc[:, 0] # all rows, first column
df.loc['Ali'] # row by label
df.loc['Ali', 'Salary'] # single cell
df.loc['Ali', 'Salary'] = 100 # update cell
```

Resetting & Setting Index

```
df.reset_index(inplace=True)
df.set_index('Name', inplace=True) # inplace=True: no need to reassign
```

Multi-level (Hierarchical) Index

```
big_df.loc['Simpson'].loc['Homer'] # safe chained .loc
```

3. READING DATA

```
df = pd.read_csv('data.csv')
df = pd.read_excel('data.xlsx')
df = pd.read_json('data.json')
```

4. FIRST LOOK — EXPLORATORY FUNCTIONS

Method	Description
<code>df.head(n)</code>	First n rows (default 5)

<code>df.tail(n)</code>	Last n rows (default 5)
<code>df.info()</code>	Column types, non-null counts, memory
<code>df.describe()</code>	Stats: mean, std, min, max...
<code>df.shape</code>	Tuple (rows, columns)
<code>df.columns</code>	List of column names
<code>df.dtypes</code>	Data type of each column
<code>df.count()</code>	Non-null count per column
<code>df.isna().sum()</code>	Count of NaN per column

5. HANDLING MISSING DATA (NaN)

```
df.dropna() # drop rows with any NaN df.dropna(axis=1) # drop columns with any NaN
df.drop('Paris', axis=1) # drop a specific column df.fillna(0) df.fillna(df['Paris'].mean()) #
fill with column mean df.fillna(method='ffill') # forward-fill
```

6. FILTERING & SORTING

```
df[df['Salary'] > 3000] df[(df['Salary'] > 3000) & (df['Age'] < 35)] df.query('Salary > 3000 and
Age < 35') # cleaner syntax df.sort_values('Salary', ascending=False)
df.sort_values(['Department', 'Salary'], ascending=[True, False])
```

7. GROUPBY — AGGREGATION

```
grouped = df.groupby('Department') grouped['Salary'].mean() grouped['Salary'].agg(['mean',
'min', 'max', 'count']) df.groupby(['Department', 'City'])['Experience'].mean()
df.groupby('City')['Experience'].mean()
```

8. COMBINING DATAFRAMES

Concat — Stack DataFrames

```
df_all = pd.concat([df1, df2], ignore_index=True)
```

Merge — SQL-style Joins

inner: intersection | outer: union | left: keep all left | right: keep all right

```
pd.merge(df1, df2, on='Employee_ID', how='inner') pd.merge(df1, df2, on='Employee_ID',
how='outer') pd.merge(df1, df2, on='Employee_ID', how='left') pd.merge(df1, df2,
on='Employee_ID', how='right')
```

9. APPLY() & LAMBDA FUNCTIONS

`apply()` runs any Python function row-by-row or column-by-column — great for custom transformations.

```
def salary_category(x): return 'High' if x > 5000 else 'Low' df['Category'] =
df['Salary'].apply(salary_category) def adjust_score(row): if row['Experience'] > 10: return
row['Performance_Score'] + 1 return row['Performance_Score'] df['Adjusted_Score'] =
df.apply(adjust_score, axis=1) # Lambda — inline one-liners df['Name'] =
df['Name'].apply(lambda x: x.replace('_', ' ')) df['Salary_K'] = df['Salary'].apply(lambda x:
```

```
round(x / 1000, 1))
```

10. USEFUL EXTRAS

```
df.duplicated().sum() df.drop_duplicates(inplace=True) df['Col'].value_counts() # frequency of
each unique value df['Col'].nunique() # number of unique values df.rename(columns={'OldName':
'NewName'}, inplace=True) df['Col'] = df['Col'].astype('int') df['Date'] =
pd.to_datetime(df['Date']) df.to_csv('output.csv', index=False) df.to_excel('output.xlsx',
index=False)
```